

Installed, Launched, Probed: An Execution-Based Conformance Census of the Model Context Protocol Registry

Anonymous Author(s)

Abstract—The Model Context Protocol (MCP) connects LLM agents to external tools, and public registries now list tens of thousands of MCP servers. Most empirical studies of this ecosystem are *static*: they scan source code, mine issue trackers, or read registry metadata. None calls the tools of registry servers at scale to check whether the servers start and follow the protocol. This paper does. From the official registry (17,432 unique servers) we run a complete census of all 6,089 self-contained, locally-installable servers, each launched in an isolated sandbox and driven through a conformance harness. The harness checks the initialization lifecycle and declared-schema validity across every tool a server lists, then sends an unknown tool name, an argument of the wrong type and a malformed protocol frame. Only 61.2% (95% CI 57.7–64.4) of installable servers start and complete a handshake, and 88.4% (95% CI 87.3–89.4) of those that respond report an unknown tool the same way, which differs from the error reporting the specification illustrates. We trace this to SDK defaults, and an open specification proposal would make it the specified behavior. 3.5% (95% CI 3.0–4.1) execute a tool on an argument their own declared schema rejects, although the specification requires servers to validate all tool inputs. The result is well-formed, and a client cannot tell it apart from a correct answer. Both behaviors come from the SDKs and can be measured and fixed there. We release the harness with the disclosure-filtered dataset.

I. INTRODUCTION

Large language model agents act through external tools, and the Model Context Protocol (MCP) is the dominant standard for connecting them. In under two years MCP went from one vendor’s proposal to an ecosystem whose public registries list tens of thousands of servers. One of them, the official registry, contains 17,432 unique servers at the time of our study. When a server fails to start, accepts ill-typed arguments or returns a malformed result, the agent calling it can act on wrong output without knowing it. Every agent built on MCP inherits the reliability of this server layer.

Most published empirical studies of the MCP ecosystem examine servers *without running them*, and those that do run them look for vulnerabilities. Existing work scans source code and metadata for health and maintainability, crawls registries to characterize scale and supply-chain structure, or mines GitHub issues to build fault taxonomies. None of it answers a simpler question: *if you install these servers and speak the protocol to them, do they start and do they follow the specification?* The protocol’s own conformance test suite is built for SDK maintainers to run in continuous integration, and we know of no use of it on the deployed ecosystem. It

exercises the SDKs, and the 17,432 servers published on top of them go untested.

We ran that measurement. Starting from the official registry, we census every self-contained, locally-installable server, launch each in an isolated sandbox, and drive it through a conformance harness. The harness exercises the initialization lifecycle, tool listing and declared-schema validity. It then sends three inputs that come up in ordinary agent sessions: a call to an unknown tool, a call with a wrong-typed argument, and a malformed protocol frame. Every check covers a requirement that all observed revisions state in the same terms, and the version each server negotiates is recorded with its result. Requirements stated with an RFC-2119 keyword (MUST/SHOULD) are graded apart from behavior the specification only shows in an example.

Only 61.2% (95% CI 57.7–64.4) of installable servers get as far as a completed protocol handshake. The rest fail in ways we classify into a startup-failure taxonomy, from install errors to silent exits to hangs. Among servers that do respond, 88.4% (95% CI 87.3–89.4) report a call to a non-existent tool as a “successful” tool-execution error (`isError: true`) instead of the JSON-RPC protocol error the specification illustrates. We trace this divergence to the default behavior of the official SDKs, which is inherited by the servers built on them. An open proposal would reclassify unknown-tool failures to match what the SDKs already do [1], and our census measures how much of the ecosystem already behaves that way. Separately, 3.5% (95% CI 3.0–4.1) of responding servers run a tool on input that violates the schema they publish, and return a well-formed result. In one case a tool that scans code for package imports was handed an integer and replied `CLEAN`, with neither an error code nor an `isError` flag a client could act on. The specification requires servers to validate all tool inputs. A further 0.5% (95% CI 0.3–0.7) fail to recover from a malformed frame and answer nothing further, although JSON-RPC 2.0 requires a reply to every request.

We organize the study around five research questions:

- **RQ1 (Runnability).** What fraction of registry-published servers can a client install and start, and when they do not start, why not?
- **RQ2 (Lifecycle conformance).** Of the servers that respond, how many implement the initialization lifecycle correctly, list their tools and publish valid schemas?
- **RQ3 (Input enforcement).** How many servers run a tool whose declared schema rejects the argument it was given,

and what does the client receive when they do?

- **RQ4 (Specification versus practice).** Where the specification and deployed behavior disagree on error reporting, how large is the divergence, what causes it, and which side is moving?
- **RQ5 (Correlates).** Do runnability and conformance vary with the package registry a server ships on?

This paper makes four contributions:

- **A protocol-level conformance measurement of the MCP ecosystem by execution** (C1): prior ecosystem studies read code and metadata. We install every eligible server in the registry, speak the protocol to it, and test how it responds to inputs.
- **A two-dimensional taxonomy** (C2) of startup-failure classes and conformance-violation classes, each tied to a concrete agent-facing consequence.
- **A root-cause account of the spec-practice divergence** (C3): by attributing each server to its SDK, we show that the dominant divergence comes from defaults in the official SDKs.
- **Open infrastructure** (C4): a conformance harness that extends the official suite’s scenarios to arbitrary deployed servers, plus the released measurement dataset. Others can repeat the census, and authors can check their own servers.

II. BACKGROUND AND RELATED WORK

A. The Model Context Protocol

MCP standardizes how LLM agents discover and invoke external tools. A client and server negotiate a protocol version and capabilities during an `initialize` handshake. The client can then list tools (`tools/list`) and invoke them (`tools/call`). Messages follow JSON-RPC 2.0, including its error-object semantics and its reserved error codes. Most deployments run the server locally over `stdio`, and on that transport `stdout` carries protocol traffic and nothing else. The specification is versioned by date, and servers may negotiate down to a version they support.

B. Static studies of the MCP ecosystem

The ecosystem has been studied at scale but rarely by running it, and the two peer-reviewed studies of this ecosystem are both static. One analyzes source and metadata for roughly 1,900 GitHub-sourced servers with static scanners and code-smell detectors [2]. The other surveys the protocol’s ecosystem and its security threats [3]. Ours differs on both axes: the population comes from the official registry a client resolves against, and every server is executed.

Registry measurement elsewhere crawls several marketplaces and characterizes scale, maintenance and supply-chain concentration across some 8,000 servers, using marketplace data without running any of them [4]. A companion dataset effort catalogues 3,238 candidate MCP repositories on GitHub and classifies them by operational role, again from repository evidence [5], and a parallel line studies the *application* side, characterizing how 1,723 MCP applications configure and

supervise the servers they consume [6]. Fault taxonomies take a third route and mine GitHub issues [7], [8]. One finds tool faults and data-schema faults the two most frequent categories [7], the same two areas our harness tests. None of this work executes a server.

C. Execution-based work is security-focused

Some prior work does execute MCP servers, to find security problems. Sandboxed behavioral scanners hunt vulnerabilities in curated corpora [9], and taint-style analyzers confirm exploitable flaws across tens of thousands of repositories [10]. A security-invariants benchmark avoids live third-party execution altogether in favor of fixtures and a 40-repository metadata sample [11]. Malicious-server detection, authentication studies and dynamic assessment of internet-facing endpoints all keep to security questions [12], [13], [14].

MCPZoo assembles 64,611 unique servers, of which more than 37,288 support dynamic analysis, and uses them to show that existing security scanners are unreliable, with fewer than half of sampled alerts holding up under manual validation [15]. That corpus differs from ours in two ways. First, MCPZoo is built by *repairing* servers: a multi-agent framework infers environments and iteratively fixes deployment and runtime defects until in-the-wild repositories become running services. That construction is the right choice for studying what a server does once it runs, and it necessarily removes the signal we measure: how often a server runs when a client installs it as published. Second, its population is collected from multiple public markets, and ours is the official registry, the index a client installs from. That their corpus needed a repair framework fits our runnability finding. Conformance is a separate question: whether a server that is neither malicious nor vulnerable does what the specification says. Concurrently with our work, Afsar draws a seeded random sample of 400 npm `stdio` servers from the same registry and probes each without repair. There 48.8% complete a handshake, servers that never start (37.5%) outnumber those missing credentials (13.3%), and the 195 that run show no fatal schema violations across 2,766 tools [16]. That study samples one package ecosystem and checks declared structure. We census both npm and PyPI and test behavior, calling tools with inputs the server should reject.

D. The specification’s own error-handling debate

The categorization this paper measures is under active revision. SEP-1303, accepted and reflected in the current specification, moved tool input validation errors out of protocol errors and into tool execution errors, reasoning that a language model can self-correct only from an error that reaches its context [17]. SEP-2145, still open, extends the same treatment to tool resolution failures including unknown tools. Its stated purpose is alignment with what the Python and TypeScript SDKs already do [1]. We take no position on which mechanism is correct. We measure what fraction of the deployed ecosystem the proposed alignment already describes. Concurrent with this work, a practitioner census

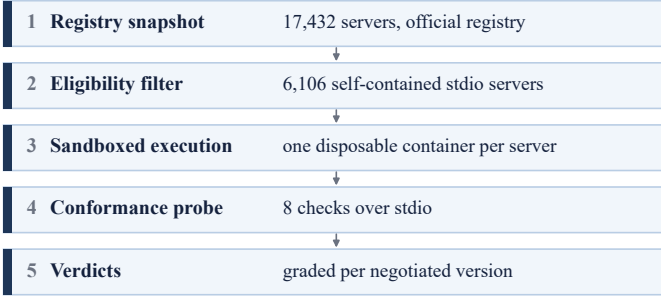


Fig. 1. Measurement pipeline. Every eligible server is installed and executed in its own disposable container, then driven through the conformance probe and graded against the protocol version it negotiates.

posted to the same proposal probed roughly 10,500 *remote* registry endpoints and harvested their tool lists. It reports that 88.8% of mutating tools declare no output schema, and argues that the proposal’s output-validation clause is inert for most of the write path [18]. That measurement and ours are close to disjoint: it probes hosted HTTP endpoints and reads declared metadata while we install and execute local stdio servers and test behavior against inputs. It addresses the output-validation clause, and our checks cover tool resolution and input enforcement.

E. Conformance tooling and the gap we fill

The protocol maintainers ship a conformance suite for client and server implementations, aimed at SDK authors running it in continuous integration. To our knowledge it has not been used to measure the deployed ecosystem. As far as we know, ours is the first reproducible conformance measurement of this ecosystem that installs and executes the published servers and tests their behavior against adversarial inputs with an open harness and dataset.

III. METHODOLOGY

We measure the MCP server ecosystem by *executing* it. This section defines the sampling frame, the conformance harness, the sandbox, and the grading rules (Figure 1). The harness and the released dataset make every number reproducible.

A. Sampling frame

We crawl the official MCP registry (`registry.modelcontextprotocol.io`) via its cursor-paginated API, retaining the latest published version of each server, which yields 17,432 unique servers. Each server declares zero or more *packages* (installable artifacts on npm, PyPI, OCI, NuGet, or Cargo) and/or *remotes* (hosted HTTP endpoints). Our study population is the set of servers that are (i) locally installable, (ii) use the `stdio` transport, (iii) distributed on npm or PyPI (the two dominant registries), and (iv) *self-contained*: they declare no required environment variable or argument that lacks a default. Criterion (iv) restricts us to servers that a client could launch without provisioning secrets, which limits what mass execution can do to third-party systems and keeps

the population reproducible. We report the exclusion of remote-only and credential-requiring servers (Section VI) and treat the resulting frame as the denominator for all rates. We probe the *entire* frame (all 6,089 eligible servers), which makes the measurement a census of the eligible population.

B. Conformance harness

The harness, `mcpprobe`, is a minimal, hand-written JSON-RPC client speaking line-delimited messages over the server’s stdio. We do not use an official SDK client. To observe raw behavior, including responses to malformed frames, the probe must not inherit any SDK-side correction or validation. The harness offers protocol version 2025-06-18 at initialization (the then-current revision was 2025-11-25), records the version the server *negotiates*, and records it with every result. A server that supports the offered version must answer with it, and 3,576 of 3,685 responders did. The rest negotiated three other revisions: 104 settled on an older one, the oldest dating to 2024-11-05, and 5 answered with a revision newer than the one offered, which the lifecycle does not permit. Our checks encode requirements that every one of these revisions states in the same terms, so no verdict depends on the negotiated version. A requirement that moved between revisions would need a version-dependent rule, and we add none.

Each server is driven through the following checks, whose identifiers align with the categories of the official `modelcontextprotocol/conformance` suite where an analogue exists:

- **server-initialize**: the `initialize` request must return a result carrying a protocol version and a capabilities object, after which the harness sends `notifications/initialized`.
- **ping**: a `ping` must return an empty-object result.
- **tools-list**: if the server advertises the tools capability, `tools/list` must return an array in which every entry has a `name` and an `inputSchema` object.
- **tools-schema-valid**: each declared `inputSchema` must be a valid JSON Schema (checked by meta-validation).
- **tools-call-unknown**: calling a non-existent tool must be rejected. The specification’s Error Handling section categorizes “Unknown tools” under *Protocol Errors* (standard JSON-RPC errors) and illustrates this with a `-32602` example, as opposed to *Tool Execution Errors* reported as a result with `isError: true`. This categorization is prose plus an example, and it uses no RFC-2119 MUST/SHOULD keyword. A diverging response is never scored as a “violation”. Each server’s choice between the two mechanisms is recorded (see `error-as-result` below).
- **tools-call-invalid-args**: for a tool with a typed required property, calling it with a wrong-typed value should be rejected. The spec is ambiguous here, since it lists “Invalid arguments” under *Protocol Errors* but “Invalid input data” under *Tool Execution Errors*. We count *either* a protocol error *or* an `isError` result as acceptable,

and flag only a plain success response (the tool ran on ill-typed input) as a defect.

- **malformed-json**: after receiving a syntactically invalid frame, the server must not crash or hang, and a subsequent `ping` must still succeed.
- **stdout-purity**: under the stdio transport, the server must emit only protocol messages on stdout, and any non-JSON line is a channel-corruption defect.

C. Grading and the error-as-result category

Verdicts are `pass`, `fail`, `warn`, `skip`, and one category introduced by calibration: `error-as-result`. When validating the harness against the official reference servers (`server-memory`, `server-everything`), we found they answer an unknown-tool call not with the specified JSON-RPC `-32602` error but with a *successful* response whose `result.isError` flag is set. Because this behavior originates in the official SDKs, it reaches most of the ecosystem, and it earns its own verdict category. Its prevalence is a result in its own right.

D. Sandbox

Each server runs in a fresh container (`node:22-slim` for npm, an `astral-sh/uv` image for PyPI) with a memory cap, a single CPU, a PID limit, `no-new-privileges`, all Linux capabilities dropped, and no host filesystem mounts. The harness supports two probe conditions. In the **two-phase** condition, an online install pass populates the cache and the conformance probe then runs with `--network=none`, so no server code has network access while it is being measured. In the **single-phase** condition the network is available throughout, and install and probe occur together in one ephemeral container. The census reported here was measured in the single-phase condition, for the operational reasons given in Section III-E, and Section VI carries the consequences for validity.

E. Execution environment and probe condition

The census was executed on a single disposable cloud instance (no cloud credentials attached) using the online single-phase condition: install and probe occur in one ephemeral `--rm` container per server, with nothing cached between servers, which keeps disk usage bounded across the full ecosystem. The recorded launch command of every run confirms that no volume was mounted. We did not run a controlled comparison of the two conditions. In development runs on smaller subsets we saw the same error-as-result prevalence and the same silent type non-enforcement among responding servers under both. Handshake yields from those runs are not comparable, because the subsets differed.

Two groups of runs ended for reasons outside the server’s control. 32 probes (0.5%) were killed with exit 137 by either the memory cap or an automated guard that reclaims host disk space, and the census does not record which. Another 17 (0.3%) aborted on a harness-side error before any check ran. Only one of these 49 runs had completed a handshake first, and the rest can only lower the reported yield.

F. SDK-level confirmation

Where a conformance failure concentrates in one SDK family, the census cannot say whether the library permits the failure or the authors introduced it independently. For the input-validation result we probed the SDK on its own. We build two minimal servers, each exposing one tool that declares a single required string property. One uses the low-level `Server` API and the other the high-level `McpServer` API, both on the same SDK release. Each is driven with the harness check used in the census: a `tools/call` carrying an integer where the declared schema requires a string. The construction is otherwise identical, so any difference in outcome is a property of the API path. We report the result in Section IV-C, and the two servers and the probe script are released with the harness.

G. Funnel accounting

RQ1 asks whether a server runs, and we count every stage: declared $\rightarrow$ eligible (self-contained stdio npm/PyPI) $\rightarrow$ container started $\rightarrow$ process launched $\rightarrow$ handshake completed $\rightarrow$ probed. Servers that never complete a handshake are classified into startup-failure categories (Section IV) from their exit code and `stderr`, which accounts for the lost yield. Runs broken by the harness are a separate outcome, reported with the funnel and excluded from the graded population, because they measure our instrument.

IV. RESULTS

We launched all 6,106 servers in the eligible frame, and on 17 of them our own probe failed on a broken pipe or an uncaught exception, leaving those runs with no verdict about the server. We report them with the funnel and exclude them from every denominator, which leaves 6,089 graded servers.

A. RQ1: Runnability and the startup funnel

3,729 of 6,089 servers (61.2% (95% CI 57.7–64.4)) complete the MCP handshake once the entry-point artifact described in Section IV-E is corrected, against an uncorrected census figure of 60.5% (95% CI 59.3–61.7). Roughly two in five never serve the protocol, and Figure 2 classifies those failures by cause.

Servers that only need credentials or configuration are counted apart from servers that crash, since a missing setting is a documentation gap. Most failures happen *after* a successful install: the package downloads, then the process crashes, hangs, or exits without speaking the protocol. The software fails after the packaging has done its job. A concurrent random draw of 400 npm servers reports a 48.8% handshake rate [16], against 66.2% for the npm servers in our census. Snapshot and harness both differ between the two studies, which is enough to explain a gap of that size.

B. RQ2–RQ3: Conformance and robustness

Table II reports the verdict distribution for each check over the 3,685 responding servers, with 95% Wilson confidence intervals. Lifecycle and tool-listing conformance are high, and

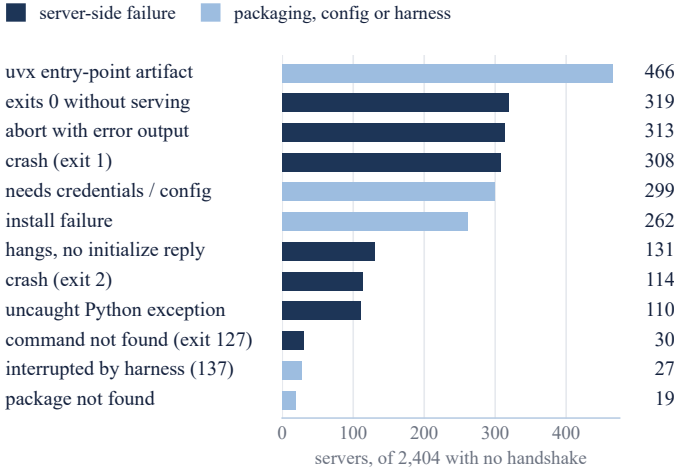


Fig. 2. Why servers never reach a handshake. Configuration requirements and the entry-point artifact are separated from true startup failures.

TABLE I
STARTUP-FAILURE TAXONOMY: CAUSES FOR SERVERS THAT NEVER REACH A HANDSHAKE.

Failure class	Count	% of failures
entrypoint-not-provided	466	19.4
exit-silent	319	13.3
crash-with-error-output	313	13.0
crash-exit-1	308	12.8
needs-auth-or-config	299	12.4
install-error	262	10.9
hang-no-reply	131	5.4
crash-exit-2	114	4.7
crash-python-exception	110	4.6
crash-exit-127	30	1.2
crash-exit-137	27	1.1
install-not-found	19	0.8
unclassified	2	0.1
crash-exit-255	1	0.0
crash-exit-3	1	0.0
crash-exit-4	1	0.0
crash-exit-254	1	0.0
Total non-handshaking	2404	100.0

the discriminating checks are error handling, argument type enforcement, and malformed-input survival. 7.4% (95% CI 6.6–8.3) of responders failed to reject a wrong-typed required argument, which Section IV-C decomposes into the servers that reported the problem anyway and the smaller class that did not. 0.5% (95% CI 0.3–0.7) failed to recover from a malformed frame, which we classify as a robustness and conformance failure. The security reading does not survive the transport: under stdio the client is the parent process and the sole writer to the server’s standard input, and a party able to deliver a malformed frame can already terminate the subprocess. JSON-RPC 2.0, on which MCP is built, nonetheless defines a parse-error response for this case and requires a reply to every subsequent request [19], and these servers give neither. One maintainer who reproduced the finding made this argument, and the classification here follows it (Section VI).

TABLE II
VERDICT DISTRIBUTION PER CHECK OVER RESPONDING SERVERS, WITH 95% WILSON CIs ON THE PASS (OR DOCUMENTED) RATE.

Check	Pass	Fail	Other	Pass rate (95% CI)
server-initialize	3685	0	0	100.0 (99.9–100.0)
ping	3638	47	0	98.7 (98.3–99.0)
tools-list	3673	5	7	99.7 (99.4–99.8)
tools-schema-valid	3671	2	0	99.6 (99.4–99.8)
tools-call-unknown	139	166	3380	3.8 (3.2–4.4)
tools-call-invalid-args	3309	273	103	89.8 (88.8–90.7)
malformed-json	3668	17	0	99.5 (99.3–99.7)
stdout-purity	3654	31	0	99.2 (98.8–99.4)



Fig. 3. The coarse verdict splits in half. Only the left segment is undetectable by the client: the tool ran on input its schema rejects and returned a well-formed result carrying no error indication.

C. RQ3 refined: what a failure to reject returns to the client

The `tools-call-invalid-args` verdict records whether a server rejects an argument that violates its own declared schema. As a protocol judgment that is correct, but it cannot tell a client what it received, because two different behaviors produce the same verdict. A server may validate the input and report the problem in the result *text* without setting `isError`, which is non-conformant but leaves the information available. Or it may execute the tool and return ordinary-looking output, which does not. We separate the two by classifying the recorded response of every server in this category (Figure 3, Table III).

Of 273 such servers, 118 (43%) reported the problem in the result text. The server noticed, and a client willing to parse prose could too. 129 (47%) executed the tool and returned a result with no indication that anything was wrong. For 26 the response could not be classified from the transcript. The hazard class is 3.5% (95% CI 3.0–4.1) of responding servers, smaller than the 7.4% (95% CI 6.6–8.3) of the raw verdict, and we report the smaller figure.

Splitting the two takes a classifier, and ours is a keyword rule over an English error vocabulary. To measure what it gets wrong, a human rater labeled a stratified sample of 40 responses against a codebook fixed before any item was read, with the classifier’s answer hidden. The two agreed on 34 of 40 (Cohen’s $\kappa = 0.70$).

The errors are asymmetric. Of the responses the classifier calls silent execution, the rater agreed on 95% (19 items), giving a precision of 95%. Of those it files as reporting the problem in prose, the rater found 24% were tools that had run anyway (21 items), giving a recall of 78%. Reweighting each stratum by its size in the census puts the true count near 150 rather than 129, in a range of 110 to 181. The keyword rule under-counts, and 129 is the conservative figure. We report

it as the headline and treat the corrected range as the better estimate of the true rate.

The specification’s security requirements state that servers **MUST** validate all tool inputs [20]. A server that executes a tool on an argument contradicting its own published schema has not validated that input. These 129 servers fail an explicit RFC-2119 requirement, as do the servers that write non-protocol output to stdout, which the stdio transport forbids with a **MUST NOT**.

Here SDK attribution narrows the cause further than it does for the error-reporting divergence. 114 of the 129 are built on the official TypeScript SDK and 0 on either Python SDK, against responding populations of 2,449 and 1,027 respectively. A behavior absent from one language ecosystem and concentrated in another points to the library, and we went on to test the SDKs (Section III-F). On @modelcontextprotocol/sdk 1.30.0, the current release at the time of writing, a server built with the low-level Server API publishes an `inputSchema` in `tools/list` and the SDK then does not check `tools/call` arguments against it. A tool declaring a string property, called with an integer, runs and returns a well-formed result. The same tool built with the high-level `McpServer` API, whose schemas carry type information the SDK already validates, rejects the identical call with an `isError` result. We did not test the Python SDKs under the same conditions, and their absence from this population stands as an observation only. Within the TypeScript SDK the gap is that one supported construction path publishes a contract the library never enforces. The servers on that path have not declined a check the SDK offered them. Of the 114 TypeScript-SDK servers, 111 declare a caret range and none declares an exact version. Because each server was installed fresh, those ranges resolve to the newest 1.x. The behavior is current, and a fix on the 1.x line would reach almost all of these servers at their next install.

The unknown-tool divergence has a merged fix that deployed servers do not show. The SDK’s tracker records tool resolution as corrected in March 2026, four months before our crawl. The change moves the tool-not-found check outside the handler’s `try` block, letting it propagate as a protocol error [21], [22]. We rebuilt the minimal high-level server on releases 1.28.0, 1.29.0 and 1.30.1 and sent the same unknown-tool call. All three answer with an `isError` result whose text reads `MCP error -32602: Tool ... not found`. The protocol error is raised and then delivered inside a successful response, where a client branching on JSON-RPC errors cannot see it. This explains a puzzle in our own data: the census ran after the fix shipped, and the rate did not move.

The low-level path loses the tool name as well, which we had not tested. That API takes one handler for every `tools/call` request, and the handler receives the requested name without the library checking it against the declared tool list. Our probe asked a low-level server for a tool it does not have and received that server’s ordinary answer, reporting a clean scan of code it was never given. Nothing in the response marks the tool as missing.

In the recorded responses, a code-scanning tool asked to scan the integer 12345 answered CLEAN. A prompt-scoring tool returned a complete rubric with per-dimension sub-scores for the same integer, and a project registry confirmed that a project named 12345 had been registered. All three are well-formed successful results, and none answers what was asked. At the protocol level none can be told apart from a correct answer: there is no error code and no `isError` flag. Each server published a schema that would have caught the call and then left it unenforced.

D. RQ4: Where practice and specification disagree, and which one moved

88.4% (95% CI 87.3–89.4) of responding servers answered a call to a tool that does not exist with an `isError` tool-execution result instead of a JSON-RPC protocol error. The specification’s error-handling section lists “unknown tool” under *Protocol Errors* and illustrates it with a `-32602` example [20], and attaches no RFC-2119 keyword. The harness logs which mechanism each server uses, and neither is scored as a violation (Section VI).

The specification has been moving toward what servers do. SEP-1303, now accepted and reflected in the specification, reclassified input validation errors from protocol errors to tool execution errors on the grounds that a model can only self-correct from an error it can see [17]. SEP-2145, open at the time of writing, proposes the same move for tool resolution failures, covering unknown and disabled tools. Its stated rationale is that the change “aligns specification with existing Python and TypeScript SDK behavior” [1]. We can say how much of the deployed ecosystem that alignment already describes: 88.4% (95% CI 87.3–89.4) of responding servers. (Aggregate figures from this census were posted to the proposal’s discussion thread while it was open.) The other 11.6% do not form one population: 139 servers emit a protocol error coded `-32602` or `-32601`, 122 use some other code, and 166 return a plain success that states the failure only in its text, a mechanism neither proposal describes.

To explain the uniformity, we attribute each server to its SDK from its resolved dependency graph (Figure 4, Table IV). We take the exact package version the probe launched and resolve its runtime dependencies through `deps.dev`. A server that reaches an SDK through a third-party wrapper is credited to that SDK and not to the residual. 3,385 servers name an SDK among their own dependencies, and 100 reach one only through a wrapper. For 200 we have no graph: the package or the probed version has since been withdrawn from its registry. The behavior tracks the SDK: the official TypeScript and Python SDKs and the FastMCP framework all default to it. A small number of TypeScript-SDK servers emit the protocol error, which shows the alternative is reachable within the SDK without being its default, and the official reference servers behave the same way.

Our attribution recognizes a fixed list of known SDK package names, and the residual no-known-SDK category holds servers with none of those names anywhere in their

TABLE III

SERVERS THAT RAN A TOOL ON INPUT THEIR PUBLISHED SCHEMA REJECTS AND RETURNED ORDINARY-LOOKING OUTPUT. INPUTS ARE THE HARNESS’S TYPE-POISONED VALUES, AND OUTPUTS ARE VERBATIM BUT TRUNCATED. NONE CARRIES AN ERROR INDICATION A CLIENT COULD ACT ON.

Tool	Argument sent	Result returned to the client
scan_code_imports	code: 12345	“CLEAN — No npm package imports found in the provided code”
register_project	name: 12345	“Project 12345 registered successfully. Framework: generic”
score_prompt	prompt: 12345	a complete scoring rubric: score: 8, grade: "F", with per-dimension sub-scores
upcoming_deadlines	within_days: "not-a-number"	a dated regulatory snapshot with a populated result envelope
search	query: 12345	{"results": []}

runtime graph. Such a server may still have vendored an SDK at build time. Resolving the graph transitively moves servers that inherit an SDK through a wrapper into that SDK’s family. That leaves the residual as the sharpest contrast in the table: 47% of its 182 servers answer an unknown tool with an `isError` result, against about nine in ten in every official-SDK family. A few SDK defaults, whether used first-hand or through wrappers, account for this behavior across the ecosystem, and where they are absent so is the behavior.

a) *A third SDK, found inside the residual:* A maintainer’s report gave us a way to test that claim on a library we never sampled as a family. Their server failed our malformed-frame check and contains no framing code at all: it is built on the official Go SDK, whose stdio transport ends the session on a decode error (Section VI). Our attribution had recorded it as having no known SDK, because a Go binary published through npm declares no MCP dependency. Reading each residual server’s repository language finds 11 such Go servers, all of which complete a handshake.

They behave as a family, and not as the rest of the population does. 5 of the 11 fail the malformed-frame check, a rate of 45% against 0.5% (95% CI 0.3–0.7) across all responders. On the unknown-tool check every one of the 11 returns the JSON-RPC protocol error, where 88.4% (95% CI 87.3–89.4) of responders return `isError`. Under the population rate for that response, eleven consecutive protocol errors would arise by chance with probability below 10^{-15} .

Two independent checks, opposite defaults, one library. This is the same claim the error-as-result result makes, tested on a library that reached our census only as a compiled binary, and it is the cleanest evidence we have that these verdicts record an SDK rather than an author. The set is small and was identified by repository language rather than by dependency graph, so we report it as a confirmation of the mechanism and not as a rate for the Go ecosystem.

E. RQ5: Correlates

We test whether runnability and conformance correlate with the package registry (Figure 5). Runnability differs: npm servers complete a handshake much more often than PyPI

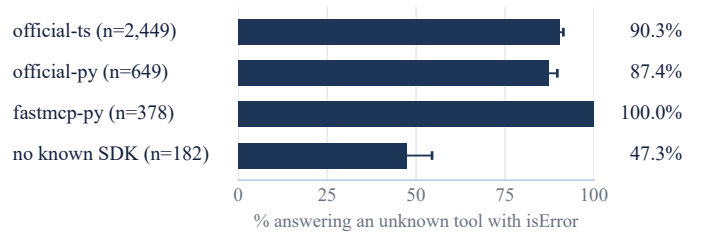


Fig. 4. Unknown-tool responses by SDK family. In every family, most servers answer with an `isError` result. Bars show 95% Wilson intervals, and families with fewer than 25 servers appear in Table IV only.

TABLE IV
SDK FAMILY VS. UNKNOWN-TOOL RESPONSE. THE MECHANISM A SERVER USES IS INHERITED FROM ITS SDK, AND FEW AUTHORS CHANGE THE DEFAULT.

SDK family	error-as-result	protocol error	other
official-ts	2211	65	173
official-py	567	0	82
fastmcp-py	378	0	0
no-known-SDK	86	63	33
metadata-unavailable	16	2	0
fastmcp-ts	0	8	0
xmcp-ts	0	1	0

servers (66.2% of 3,952 against 50.1% of 2,137, a difference of 16.1 pp (95% CI 7.2–23.9)). Conformance does not: the same publisher-cluster bootstrap puts the error-as-result difference at -2.5 pp (95% CI -6.0–1.6), an interval spanning zero. Both comparisons use the same method, and neither rests on a stronger footing than the other. The pattern follows from RQ4, where the divergence is inherited from the SDKs and both language ecosystems’ SDKs share the same default. The registry a server ships on changes whether it starts but not whether it conforms.

Part of this gap came from our own launcher, and we corrected for it. We launch PyPI servers as `uvx <package>`, which resolves only if the package’s console script is named after the package. npm’s `npmx` resolves the entry point from



Fig. 5. Handshake yield (PyPI after the entry-point correction) and the error-as-result rate for unknown tools, by package registry, with 95% intervals from a publisher-cluster bootstrap.

`package.json` and has no such requirement. 462 PyPI servers (21.6% of the PyPI population) installed successfully but were never launched for that reason and no other. Our classifier identifies 466 such rows in the census, the completed re-probe covers 462 of them, and 4 were missed. We re-probed those servers with the entry point resolved from the installed package metadata. 44 (9.5% (95% CI 7.2–12.5)) then completed a handshake, and the rest failed for reasons unrelated to naming (Python exceptions, hangs, undeclared configuration). Correcting for this raises PyPI runnability from 50.1% to 52.1% (95% CI 46.3–59.1) and the overall census figure to 61.2% (95% CI 57.7–64.4), which we report as the headline throughout. The npm–PyPI gap narrows from 16.1 to 14.0 percentage points but stays large.

The entry-point problem is also a finding about the ecosystem. Registry metadata does not record a server’s entry point, and a client working from the registry and nothing else cannot launch a fifth of published PyPI servers. Once the entry point is resolved, nine in ten of those servers still fail to start, and a listing in the registry says nothing about either outcome.

V. DISCUSSION

a) The specification and the SDKs: For whoever decides SEP-2145, accepting it would mostly document practice that already exists. SEP-1303 reclassified input validation failures as tool execution errors, arguing that a model can only correct an error it sees [17], and SEP-2145 proposes the same for unknown tools [1]. 88.4% (95% CI 87.3–89.4) of responding servers already return `isError` for an unknown tool. Rejecting the proposal would leave nine in ten responding servers non-conformant by example, and we do not expect 3,258 servers to be changed by hand. The remaining servers do not form one alternative either: 139 emit a protocol error, 122 emit a different code, and 166 report the failure only in the text of a successful result.

The behavior tracks the SDK rather than the author. The official TypeScript and Python SDKs and the third-party wrappers built on them all default to `isError`. So 88.4% (95% CI 87.3–89.4) should not be read as consensus among server authors. It measures SDK adoption. The practical consequence is that the lever is a small number of libraries, and a change

there reaches thousands of servers at once. The limit of that lever is visible here too: the unknown-tool fix recorded on the TypeScript SDK’s tracker in March 2026 did not move the rate, and the error still arrives inside a successful response.

b) Silent execution is a different kind of finding: The unknown-tool case is a disagreement about the error channel, and the client is still told the call failed. Here the client is told nothing. Input validation is an explicit MUST in the specification with no pending proposal to relax it, so this cannot be framed as the specification lagging practice. 114 of the 129 are built on the official TypeScript SDK, whose low-level `Server API` publishes a tool’s `inputSchema` and does not enforce it (Section IV-C). That looks like a library gap rather than 129 separate author mistakes.

The client can no longer tell the result apart from a correct one. There is no error code, no `isError` flag, nothing to branch on. `CLEAN`, returned for a scan of the integer 12345, is indistinguishable from a real clean scan, so there is no retry, no argument repair, and no warning to the user. 129 is a lower bound: we counted any error-like token in a reply as a report.

c) Startup failure and what a registry could do about it: Being published does not mean it runs, and the registry carries no signal for it. Pimentel et al. report the same for notebooks [23], as do Ibrahim et al. for images that share a name on Docker Hub [24]. The registry should attempt a handshake at publish time and show the outcome: flag it, not delist it. It should also record the entry point and the required configuration. 462 PyPI servers could not be launched from registry data alone, and 299 failed for no reason other than undeclared configuration. We ran this check across 6,089 servers, so it is feasible at registry scale.

d) What running the servers shows that reading them cannot: Studies that analyze MCP servers from source or metadata [2], [3] cannot observe any of these failures. Observing them needs execution. In line with Ait et al. on Hugging Face Hub [25], the registry as a data source is also less reliable than it looks: our own launcher undercounted PyPI by about two percentage points until we corrected the entry-point problem (50.1% uncorrected against 52.1% (95% CI 46.3–59.1) after).

The two package registries separate the two questions. Runnability differs by 14.0 percentage points, 66.2% against 52.1% (95% CI 46.3–59.1), while conformance among the servers that do run does not. Packaging is per-ecosystem. Protocol behavior is per-SDK. Improving conformance means fixing a few libraries and improving runnability means fixing thousands of packages, which are very different amounts of work.

e) A moving specification: The lifecycle we measure is being retired. Revision 2026-07-28 drops the initialization handshake and negotiates a version on every request instead, with an optional `server/discover` call for clients that want to ask up front [26]. Handshake-based behavior is now described as a backward compatibility concern. Our population sits well behind that boundary: 97% of responders settled on 2025-06-18, and 104 answered with something older still. This

census records an installed base at the point its mechanism left the specification, and measures the distance between the two. A server published today inherits whichever default its SDK shipped with, and the revision that removes the handshake reaches those servers only when their authors upgrade, which the version spread here suggests is slow.

f) Recommendations: The specification should accept SEP-2145, state it with an RFC-2119 keyword instead of an example, and add the unknown-tool case to the conformance suite so that it is tested.

SDK authors should validate `tools/call` arguments against the declared `inputSchema` by default, in the low-level Server API as well, and check the tool name against `tools/list`. Most authors never change the default, so the default is the behavior.

Agent framework authors should not treat a result without `isError` as correct. They should validate arguments client-side against the schema before the call, and treat a server as untrusted until it completes a handshake.

Registry maintainers should run a handshake at publish time, show the result, and store the entry point and the required configuration in the listing.

VI. THREATS TO VALIDITY

a) Construct validity: Our verdicts operationalize “conformance,” and a poorly chosen operationalization could over- or under-count. Our checks cover requirements that every revision in the census states in the same terms, which keeps any server from being judged against a rule that postdates the revision it negotiated, and we also distinguish normative requirements from illustrative ones. The most prevalent divergence we report (unknown-tool responses) is graded in its own **error-as-result** category and never counted as a failure, because the specification expresses it only in prose and an example, without an RFC-2119 keyword. We calibrated the harness on the official reference servers before the study and confirmed it reproduces their known-good behavior on every check except the SDK-level divergence we set out to measure.

b) Internal validity: The census ran in the single-phase condition, with the network available during both install and probe (Section III-E). This avoids penalizing a server that legitimately contacts the network during initialization, but it does not remove server-side outbound calls as a confounder. The exposure is concentrated in one check: for `tools-call-invalid-args` we accept an `isError` result as evidence that the server rejected the ill-typed argument. An `isError` produced instead by an unrelated runtime or network failure is indistinguishable to the harness, and would be counted as argument checking that did not in fact occur. The rating pass below puts a number on that: a quarter of the responses the classifier read as argument checking were, to a human reader, tools that had run anyway.

A second artifact affects the same check. Where a tool declares several required properties, the harness poisons one and fills the rest with placeholder values, so a server may reject a placeholder and never examine the argument under

test. We find one such response in the 273 we classified, which bounds the effect at well under a percentage point, and a re-run records which property was poisoned so the case can be excluded directly. The lifecycle, tool-listing, error-handling, malformed-input and stdout-purity checks are local protocol behaviors and are not exposed to this confound. A network-isolated replication would tighten the argument-checking result, and the harness already supports that condition. Sandbox heterogeneity across language runtimes is a further source of noise. We bound it by launching each package at the version its registry entry declares and by recording every run’s full launch command, including the base-image tag.

c) External validity: Our population is self-contained, locally-installable servers distributed on npm and PyPI using the stdio transport. This excludes remote-only servers (which we cannot execute without provisioning hosted infrastructure) and servers requiring credentials (which we exclude for both ethics and reproducibility). We report the size of each excluded class and confine all rates to the stated frame. Whether remote and credentialed servers behave the same way is untested here. Agent clients mostly launch local stdio servers, and we studied this population first.

d) Population counts and publisher clustering: Because we census the eligible frame, the counts are population counts. The intervals we report do not describe sampling error. They describe how far a rate would move if the registry had drawn a different set of publishers, which is the uncertainty that matters when a single publisher ships hundreds of servers from one template. The 6,089 servers in our frame come from only 3,899 distinct registry namespaces, and the distribution is heavy-tailed: the largest single publisher accounts for 305 servers (5.0% of the census) and the ten largest for 12.6%. Servers from one publisher frequently share a template and behave near-identically; a binomial interval that treats them as independent trials is therefore too narrow. Every interval in this paper comes from a bootstrap over publishers rather than over servers. The widening is substantial: the handshake interval is 3.2 times wider than the binomial one (60.5% (95% CI 56.2–64.0) against 60.5% (95% CI 59.3–61.7)). Error-as-result becomes 88.4% (95% CI 86.6–90.0) and silent type non-enforcement 7.4% (95% CI 5.9–9.2). As a second check we recompute each rate on a subset holding one server per namespace (3,899 servers): handshake yield 58.7% (95% CI 57.2–60.2), error-as-result 85.2% (95% CI 83.7–86.6), silent type non-enforcement 8.1% (95% CI 7.0–9.3), and no conclusion changes under either treatment. The largest publisher is atypical in runnability at a 26.2% handshake yield, and its presence moves the census figure by 1.8 percentage points.

e) Validating the response classifier: The split between silent execution and an in-band report rests on a keyword rule, and an unvalidated rule behind a headline number is exactly the weakness that recent work found in MCP security scanners [15]. We therefore fixed a codebook, drew a stratified sample of 40 responses, and had a human rater label each one with the classifier’s answer hidden. Agreement was 34 of 40, Cohen’s $\kappa = 0.70$. Precision is 95% and recall 78%, so the

rule seldom invents a silent execution and regularly misses one. Reweighting by stratum size puts the true count near 150 (110 to 181) against the 129 the rule finds, and we keep the smaller figure as the headline.

Two limits on that exercise. One rater labeled the sample, so the agreement measures the rule against a single reader and not the reliability of two independent readers. An earlier pass by the same rater, used for training and released alongside the labels, is not part of this result: it was taken at speed and the tool revealed the classifier’s answer after each item, so those items were no longer blind. The sample reported here is drawn from responses that rater had not seen.

f) Severity of the malformed-frame finding, and a limitation it exposed: We initially characterized failure to recover from a malformed frame as a denial-of-service primitive. A maintainer who reproduced the finding independently argued that this overstates it, and the argument holds. Under the stdio transport only the launching client writes to that standard input, and anyone able to send it a malformed frame can already kill the subprocess. The specification also places the obligation not to send invalid messages on the client. This paper classifies the behavior as a robustness and conformance failure.

The same maintainer traced the behavior in their own server to the official Go SDK, and their report changes what this check measures. Their server contains no framing or decoding code: its stdio path is two lines, and the read loop, the JSON decoding and the decision to end the session on a decode error all sit inside the SDK. They report the behavior unchanged from v1.4.1 to v1.7.0, and they ran the same input against the other widely used Go library, `mark3labs/mcp-go`, which answers a malformed frame with a `-32700` parse error and keeps serving. On identical input the verdict is decided by which library the server links. They have filed an upstream fix [27], open at the time of writing, which would resolve it for every server built on that transport.

Their conclusion is that a per-server dataset that does not separate these is measuring the SDK rather than the server, and on this check we accept it. Like the unknown-tool divergence, `malformed-json` records a transport default that a server inherits. Another maintainer’s fix exposed a limitation in our instrument. In their server the process stayed alive while its parser absorbed the frames that followed, and the next ping timed out. Our harness records a single verdict for “did not answer after a malformed frame” and does not distinguish a terminated process from a desynchronized one. Both are conformance failures, but they are different failures with different fixes, and 0.5% (95% CI 0.3–0.7) aggregates the two. Separating them requires per-server process-state tracking that the released transcripts support but the current verdict does not encode.

That exchange also exposed a blind spot in our SDK attribution, which the same evidence lets us size. Attribution resolves npm and PyPI dependency graphs, so a server compiled from another language and shipped as a binary declares no MCP dependency and falls into the no-known-SDK residual

whatever it is built on. Of the 17 servers that fail this check, 8 are written in a language that attribution cannot resolve and 5 of those are Go. The residual therefore means “no SDK our method can see,” not “no SDK,” and its contrast with the official-SDK families in Section IV-E should be read that way. We resolve each repository’s language from registry metadata and release the table.

g) Interrupted probes: 49 probes (0.8%) ended for reasons that may lie outside the server (Section III-E): 32 killed by the memory cap or the disk guard, and 17 aborted by a harness-side error. Counting 48 of them as non-handshaking can depress the handshake yield by at most 0.8 percentage points and does not affect conformance rates. Each is identifiable in the released census rows by its exit code or its missing verdicts.

h) Reproducibility: Every number in this paper is regenerated by the released analysis scripts from the released census rows, and we have checked that the regenerated values match the ones printed here. The registry snapshot, launch commands, and package versions are recorded. Census rows do not record the harness commit, but the code that produced them is tagged `census-v1`, and the probe logic there is unchanged from the version that was calibrated.

The registry changes daily, and the object of study is the snapshot of 19 July 2026. A longitudinal study of the same registry shows how fast it moves, growing from 3,510 to 18,966 servers over 88.6 days [28].

VII. ETHICS AND RESPONSIBLE DISCLOSURE

This study executes third-party code and probes servers for defects, and it follows the norms of responsible measurement research.

a) Safe execution: Every server runs inside an isolated container during both installation and probing, with all Linux capabilities dropped, no new privileges, bounded memory, CPU and process counts, and no host filesystem access. The census was executed in the single-phase condition, where install and probe share one ephemeral container and the network is available throughout. The install phase executes arbitrary `postinstall` and `build` scripts, and it receives the same isolation as the probe. Server code also had network access while being measured, and to limit the risk every container is discarded after a single server, while the entire census ran on a disposable cloud instance with no credentials attached. Untrusted code could reach no further than one throwaway host. The harness also supports a network-isolated probe phase (`--network=none`), which we recommend for replications that do not need install and probe in a single pass. We execute no server that declares a requirement for credentials, both because it lies outside our frame and because doing so could send real requests to third-party systems.

b) Observed side effects of networked measurement: Because the census ran with the network available, we audited the captured output for evidence that server code produced effects outside its container. At least 1.3% of servers attempted an outbound connection that failed and logged an error. A

successful connection leaves no trace in the captured output, and this figure is a lower bound. Five servers emitted live credential material into their own logs (session tokens they had generated), and one auto-provisioned an API key and account against a third-party service. That last effect is the class our frame’s exclusion of credential-requiring servers was meant to prevent, and we have redacted every such value from the released dataset and asked the service to revoke the provisioned key and delete the account. Excluding servers that *declare* required credentials does not exclude servers that *acquire* them unprompted, which makes declaration-based eligibility filtering a weaker ethical guarantee than it appears. A network-isolated probe phase should be the default for measurement at this scale, and we recommend replications use the harness’s two-phase mode. We used the single-phase condition because it let one disposable host run the whole ecosystem, and the side effects above are what it cost.

c) *Responsible disclosure*: Some of our findings concern robustness. A server that stops serving after a malformed frame can desynchronize or strand a connected client, and so can a server that emits non-protocol bytes on its stdio channel. We report these only in aggregate and through anonymized case studies, and we do not publish a name-and-shame list of individual servers. Security-relevant findings are recorded in a private disclosure log with maintainer contacts resolved from registry repository metadata. We reported 15 of the 17 servers that stop serving after a malformed frame to their maintainers through their issue trackers; the remaining 2 list no repository we could reach. At the time of writing 6 of those reports are closed as resolved, and 5 of the projects have published a release since closing them. One maintainer corrected our severity assessment, and the paper follows that correction (Section VI). We also reported the input-validation gap in the TypeScript SDK’s low-level `Server` API (Section IV-C) to the SDK maintainers, who have labeled it for design work on both the 1.x and 2.x lines. Notices for the servers that run a tool on an argument their own schema rejects are prepared and going out in batches: 129 notices, which with the malformed-frame set make 146 high-severity findings in total. The remaining findings concern unknown-tool handling and stdout contamination, and they are of lower severity and appear only in aggregate.

The released dataset withholds the identities of all 321 servers carrying a security-relevant finding, replacing each name with a stable pseudonym so the rows remain linkable across files. That set includes every server in the silent-execution class. Naming a server beside a transcript of it answering a poisoned call is a claim about its author’s work. The empirical standard for secondary data asks that the reputation of the subjects be respected in the reporting of a study as well as in its conduct. Identities are restored only where a maintainer has fixed the defect or asked to be named.

d) *Registry and maintainer impact*: Our crawl pages sequentially through the registry’s public API and backs off on errors. Our installs draw each package once per probe from its public registry in the ordinary way. The released harness

lets any author check their own server’s conformance before publishing.

VIII. CONCLUSION

Earlier studies measured the MCP server ecosystem statically, or ran servers to look for vulnerabilities. We installed every eligible server in the official registry, launched each in a disposable sandbox, and tested whether it follows the protocol.

61.2% (95% CI 57.7–64.4) complete a handshake. The rest never serve the protocol, for the reasons broken down in Figure 2. Of the servers that respond, 88.4% (95% CI 87.3–89.4) report an unknown tool in the same way, and the cause is the SDKs they are built on as a direct dependency or through a third-party wrapper. An open proposal would change the specification to match this behavior, and our numbers show how much of the ecosystem it already describes. A smaller group, 3.5% (95% CI 3.0–4.1), run a tool on input their own schema rejects and return an answer with no sign that anything is wrong. One was asked to scan an integer for package imports and reported it clean, and the specification makes input validation a requirement these servers do not meet.

A static scan or a registry crawl shows none of this. The harness and the dataset are public, and others can repeat the measurement, extend it to remote and credentialed servers, or check their own servers before publishing. Because the error-reporting and input-validation behavior comes from a few SDKs, it can be changed there.

a) *Data and code availability*: The conformance harness is available at <https://anonymous.4open.science/r/mcp-conformance-F285> with the analysis pipeline and the disclosure-filtered census dataset, and the code that produced the census is tagged `census-v1`. The release contains one record per probed server (every verdict behind every number in this paper). It also carries the raw JSON-RPC transcripts and the registry snapshot that defines the sampling frame, which is everything needed to recompute the eligibility funnel and the reported rates. Two filters are applied and documented with the data. The identities of 321 servers with security-relevant findings are withheld pending disclosure, as is one server whose maintainer asked to be removed. These servers keep every measurement, but we publish neither their names nor their transcripts, and credential material that servers emitted at runtime is redacted. Neither filter alters any verdict or count, and the paper’s figures regenerate from the release unchanged. The release will be archived on Zenodo on acceptance.

ACKNOWLEDGMENT

Claude (Anthropic) is a generative AI system, and it was used extensively in this work, generating the text of every section of this paper. It wrote the measurement harness and the analysis code, including the code behind every figure, table and reported number. It also assisted with literature search, running the census and statistical analysis. Every reference and every claim attributed to a cited source was verified against that source, and every number is regenerated from the released data by the released scripts. No AI system is an author, and the authors take full responsibility for the content.

REFERENCES

- [1] Model Context Protocol contributors, “SEP-2145: Standardize tools/call failure reporting,” 2026, open proposal. <https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2145>.
- [2] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, “Model Context Protocol (MCP) at first glance: Studying the security and maintainability of MCP servers,” *ACM Transactions on Software Engineering and Methodology*, 2026.
- [3] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model Context Protocol (MCP): Landscape, security threats, and future research directions,” *ACM Transactions on Software Engineering and Methodology*, 2026.
- [4] H. Guo, Y. Hao, Y. Zhang, M. Xu, P. Lv, J. Chen, and X. Cheng, “A measurement study of Model Context Protocol ecosystem,” *arXiv preprint arXiv:2509.25292*, 2025.
- [5] B. Toepppe, A. Barrak, and E. Ksontini, “A large-scale dataset of MCP implementations on GitHub,” *arXiv preprint arXiv:2607.10123*, 2026.
- [6] M. H. A. Majeed, M. Mahmoud, and S. Nadi, “An empirical study of model context protocol applications,” *arXiv preprint arXiv:2607.25635*, 2026.
- [7] J. Owotogbe, I. Kumara, W.-J. van den Heuvel, D. A. Tamburri, A. K. Iannillo, and R. Natella, “A taxonomy of runtime faults in Model Context Protocol servers,” *arXiv preprint arXiv:2606.05339*, 2026.
- [8] M. Taraghi, M. M. Morovati, and F. Khomh, “Real faults in Model Context Protocol (MCP) software: a comprehensive taxonomy,” *arXiv preprint arXiv:2603.05637*, 2026.
- [9] Z. Tan, R. Hao, J. Singer, Y. Tang, and C. Anagnostopoulos, “MCP-SandboxScan: WASM-based secure execution and runtime analysis for MCP tools,” *arXiv preprint arXiv:2601.01241*, 2026.
- [10] P. Sun, Q. Jin, E. Huang, Z. Kang, X. Liu, D. Shen, and S. Li, “VIPER-MCP: Detecting and exploiting taint-style vulnerabilities in Model Context Protocol servers,” *arXiv preprint arXiv:2605.21392*, 2026.
- [11] T. Liu, “From tool connection to execution control: Benchmarking security invariants in MCP-style agent runtimes,” *arXiv preprint arXiv:2606.29073*, 2026.
- [12] Y. Huang, Z. Zhao, B. Chen, S. Wu, Z. Zhou, Y. Cao, X. Hu, and X. Peng, “From component manipulation to system compromise: Understanding and detecting malicious MCP servers,” *arXiv preprint arXiv:2604.01905*, 2026.
- [13] H. Zhou, X. Zhang, H. Zhang, H. Zhang, M. Zhang, and M. Yang, “A first measurement study on authentication security in real-world remote MCP servers,” *arXiv preprint arXiv:2605.22333*, 2026.
- [14] N. Padilla, “Exposed by design: A dynamic security assessment of internet-facing MCP servers at scale,” *arXiv preprint arXiv:2608.00150*, 2026.
- [15] P. Chen, B. An, M. Wu, B. Wan, G. Hong, J. Chen, X. Pan, J. Dai, and M. Yang, “Rethinking MCP security: A large-scale study of runtime MCP servers and security scanner reliability,” *arXiv preprint arXiv:2607.11086*, 2026.
- [16] H. M. Afsar, “What a random draw from the MCP registry contains, and what tool-use benchmarks contain instead,” *arXiv preprint arXiv:2609.10962*, 2026.
- [17] Model Context Protocol contributors, “SEP-1303: Input validation errors as tool execution errors,” 2025, accepted. <https://github.com/modelcontextprotocol/modelcontextprotocol/issues/1303>.
- [18] siliroid, “Registry census of declared output schemas on write tools,” 2026, comment on SEP-2145, 29 July 2026. <https://github.com/modelcontextprotocol/modelcontextprotocol/pull/2145>.
- [19] JSON-RPC Working Group, “JSON-RPC 2.0 specification,” 2013, <https://www.jsonrpc.org/specification>.
- [20] Model Context Protocol, “Model Context Protocol specification, revision 2025-06-18: Tools,” 2025, <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>.
- [21] modelcontextprotocol/typescript-sdk contributors, “fix: correct error handling for unknown tools and resources,” 2026, pull request 1389, merged 9 March 2026. <https://github.com/modelcontextprotocol/typescript-sdk/pull/1389>.
- [22] —, “Calling a nonexistent tool should trigger a JSON-RPC error,” 2026, issue 1510, closed as completed 26 March 2026. <https://github.com/modelcontextprotocol/typescript-sdk/issues/1510>.
- [23] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “Understanding and improving the quality and reproducibility of Jupyter notebooks,” *Empirical Software Engineering*, vol. 26, no. 4, 2021.
- [24] M. H. Ibrahim, M. Sayagh, and A. E. Hassan, “Too many images on DockerHub! How different are images for the same system?” *Empirical Software Engineering*, vol. 25, no. 5, pp. 4250–4281, 2020.
- [25] A. Ait, J. L. Cánovas Izquierdo, and J. Cabot, “On the suitability of Hugging Face Hub for empirical studies,” *Empirical Software Engineering*, vol. 30, no. 2, 2025.
- [26] Model Context Protocol, “Model Context Protocol specification: Versioning,” 2026, current revision 2026-07-28. <https://modelcontextprotocol.io/specification/versioning>.
- [27] Go SDK contributors, “mcp: keep the stdio session alive on a malformed JSON frame,” Pull request 1210, [modelcontextprotocol/go-sdk](https://github.com/modelcontextprotocol/go-sdk/pull/1210). Open, 2026, <https://github.com/modelcontextprotocol/go-sdk/pull/1210>.
- [28] G. Bharti, “Registry descriptions go stale unevenly: An 89-day measurement of model context protocol drift, and why drift-ranked re-auditing under-covers it,” *arXiv preprint arXiv:2608.00997*, 2026.